

```

# Import necessary libraries
from collections import deque

# Define the dimensions of the puzzle
N = 3

# Structure to store a state of the puzzle
class PuzzleState:
    def __init__(self, board, x, y, depth):
        self.board = board
        self.x = x
        self.y = y
        self.depth = depth

# Possible moves: Left, Right, Up, Down
row = [0, 0, -1, 1]
col = [-1, 1, 0, 0]

# Function to check if a given state is the goal state
def is_goal_state(board):
    goal = [[1, 2, 3], [8, 0, 4], [7, 6, 5]]
    return board == goal

# Function to check if a move is valid
def is_valid(x, y):
    return 0 <= x < N and 0 <= y < N

# Function to print the board
def print_board(board):
    for row in board:
        print(' '.join(map(str, row)))
    print("-----")

# Depth-First Search to solve the 8-puzzle problem
def solve_puzzle_dfs(start, x, y):
    stack = []
    visited = set()

    stack.append(PuzzleState(start, x, y, 0))
    visited.add(tuple(map(tuple, start)))

    while stack:
        curr = stack.pop()

        # Print the current board
        print(f'Depth: {curr.depth}')
        print_board(curr.board)

        # Check if goal state is reached
        if is_goal_state(curr.board):
            print(f'Goal state reached at depth {curr.depth}')
            return

        # Explore possible moves
        for i in range(4):
            new_x = curr.x + row[i]
            new_y = curr.y + col[i]

            if is_valid(new_x, new_y):
                new_board = [row[:] for row in curr.board]
                # Swap the tiles
                new_board[curr.x][curr.y], new_board[new_x][new_y] = new_board[new_x][new_y], new_board[curr.x][curr.y]

                # If this state has not been visited before, push to stack
                board_tuple = tuple(map(tuple, new_board))
                if board_tuple not in visited:
                    visited.add(board_tuple)
                    stack.append(PuzzleState(new_board, new_x, new_y, curr.depth + 1))

    print('No solution found (DFS Brute Force reached depth limit)')

# Driver Code
if __name__ == '__main__':
    start = [[2, 8, 3], [1, 6, 4], [7, 0, 5]]
    x, y = 1, 1

    print('Initial State:')
    print_board(start)

    solve_puzzle_dfs(start, x, y)

```

```

0 8 6
-----
Depth: 313
1 3 4
7 2 6
0 8 5
-----
Depth: 314
1 3 6
7 2 4
0 8 5
-----
Depth: 315
1 6 3
7 2 4
0 8 5
-----
Depth: 316
1 2 3
7 6 4
0 8 5
-----
Depth: 317
1 2 3
7 8 4
0 6 5
-----
Depth: 318
1 2 3
7 8 4
0 5 6
-----
Depth: 318
1 2 3
7 8 4
6 0 5
-----
Depth: 319
1 2 3
6 8 4
7 0 5
-----
Depth: 320
6 2 3
1 8 4
7 0 5
-----
Depth: 320
1 2 3
8 6 4
7 0 5
-----
Depth: 321
1 2 3
8 0 4
7 6 5
-----
Goal state reached at depth 321

```

```

class EightPuzzleIDS:
    def __init__(self, start_state, goal_state):
        self.start = tuple(start_state)
        self.goal = tuple(goal_state)
        # Define shifts for matching movements (Up, Down, Left, Right) using 1D indices
        self.moves = {
            'UP': -3,
            'DOWN': 3,
            'LEFT': -1,
            'RIGHT': 1
        }

    def get_neighbors(self, state):
        """Generates valid neighbor states and the move taken."""
        neighbors = []
        zero_idx = state.index(0)
        row, col = zero_idx // 3, zero_idx % 3

        for move, shift in self.moves.items():
            new_idx = zero_idx + shift

            # Boundary checks to prevent invalid wrapping on a 3x3 grid
            if move == 'UP' and row == 0: continue
            if move == 'DOWN' and row == 2: continue
            if move == 'LEFT' and col == 0: continue
            if move == 'RIGHT' and col == 2: continue

            # Swap blank space (0) with the target tile

```

```

        new_state = list(state)
        new_state[zero_idx], new_state[new_idx] = new_state[new_idx], new_state[zero_idx]
        neighbors.append((tuple(new_state), move))

    return neighbors

def depth_limited_search(self, current, depth, path, visited):
    """Performs a depth-wrapped DFS up to a specific limit."""
    if current == self.goal:
        return path

    if depth <= 0:
        return None

    visited.add(current)

    for neighbor, move in self.get_neighbors(current):
        if neighbor not in visited:
            # Track state history alongside the action path
            result = self.depth_limited_search(neighbor, depth - 1, path + [(move, neighbor)], visited)
            if result is not None:
                return result

    # Backtrack: Remove state so it can be evaluated via alternative paths
    visited.remove(current)
    return None

def solve(self, max_depth):
    """Iteratively increments search depth ceilings until solved or max depth reached."""
    for depth in range(max_depth + 1):
        visited = set()
        # Start path tracking with the initial layout
        result = self.depth_limited_search(self.start, depth, [], visited)
        if result is not None:
            return result, depth
    return None, -1

def print_matrix(state):
    """Formats a flat tuple state into a clean 3x3 matrix grid."""
    for i in range(0, 9, 3):
        # Format 0 as a blank space for better readability
        row_str = " ".join(str(val) if val != 0 else "_" for val in state[i:i+3])
        print(f" {row_str}")
    print()

# --- Execution ---
if __name__ == "__main__":
    # Example puzzle configuration (0 represents the blank space)
    initial_board = (2,8,3,1,6,4,7,0,5)
    target_board = (1,2,3,8,0,4,7,6,5)

    # Prompt user for the search boundary configuration
    try:
        user_limit = int(input("Enter the maximum depth limit for the search: "))
    except ValueError:
        print("Invalid input. Defaulting to a max depth of 20.")
        user_limit = 20

    print("\n--- Starting Iterative Deepening Search ---")
    solver = EightPuzzleIDS(initial_board, target_board)
    solution_path, total_depth = solver.solve(user_limit)

    if solution_path is not None:
        print(f"Solution found at depth level: {total_depth}\n")
        print("Initial State:")
        print_matrix(initial_board)

        # Iterate through actions and display matrix transformations
        for step_num, (move, state) in enumerate(solution_path, 1):
            print(f"Step {step_num}: Move {move}")
            print_matrix(state)
    else:
        print(f"A solution could not be found within your depth limit of {user_limit}.")

```

Enter the maximum depth limit for the search: 5

--- Starting Iterative Deepening Search ---
Solution found at depth level: 5

Initial State:
2 8 3
1 6 4

```
7 _ 5

Step 1: Move UP
2 8 3
1 _ 4
7 6 5

Step 2: Move UP
2 _ 3
1 8 4
7 6 5

Step 3: Move LEFT
_ 2 3
1 8 4
7 6 5

Step 4: Move DOWN
1 2 3
_ 8 4
7 6 5

Step 5: Move RIGHT
1 2 3
8 _ 4
7 6 5
```